KIET Deemed to be University

DocGuard

Document Tampering Localization Using CNN and Transformers

Deep Learning and GenAI
CS312B
B.Tech CS | Semester 5 | Section D

Team member	Roll number
Shrashti Agarwal	2400290120240
Saumya Tiwari	2400240120230
Subrat Singh	2400290120250

College Assessment Project
October 2026

Research prototype evaluated on synthetic documents. Real document accuracy has not been established.

Abstract and objectives

DocGuard is a local document tampering localization prototype. It compares a compact convolutional segmentation network with a pretrained Transformer encoder and a custom segmentation decoder. Its application accepts document images and PDF pages, highlights suspected regions and exports review evidence.

The assessment delivers a reproducible experiment and a usable review interface. The experiment uses 1,200 explicitly synthetic receipt images rather than real financial documents. Results are preliminary: the CNN achieved pixel F1 0.2460 and the fully fine tuned Transformer achieved 0.2288. High false positive rates prevent use as a reliable fraud detector.

Objectives are to localize edited regions, compare CNN and Transformer behavior under the same split, evaluate acquisition degradations and make a multilingual review workflow accessible on a CPU laptop. No GAN was trained; the implemented deep learning topic is Transformer based segmentation with a CNN baseline.

Dataset and experimental split

A generator creates 600 fictional receipt sources and an authentic and edited version of each source, producing 1,200 images with binary edit masks. There are 840 training, 180 validation and 180 test images. Source identities and layout configurations are disjoint across splits; 20 layouts are divided 14, 3 and 3.

The generator varies four fonts and six editable fields. One or two fields are changed with matched fonts, font mismatch or patch tone changes. Both labels receive comparable JPEG and blur acquisition transformations. Ground truth labels edited rectangles rather than precise character outlines.

DocTamper was considered but was not downloaded or used because access was restricted. Synthetic edits may introduce invalid dates and inconsistent arithmetic. Some matched font edits leave little visual evidence. The evaluation does not establish generalization to real documents, languages, cameras or scanners.

Model architectures

The SmallUNet baseline is a compact encoder decoder CNN with skip connections and 30,081 parameters. It learns local texture and edge patterns and produces one binary logit per output pixel. It is inspired by U Net but is not the original full architecture.

The Transformer uses the ImageNet pretrained nvidia/mit-b0 encoder with a project specific multiscale segmentation decoder. Four feature scales are projected, resized and fused into a binary mask. There are 3,349,377 parameters; the frozen encoder experiment trains 29,985 decoder parameters. This is not an unchanged official SegFormer segmentation head.

The application loads local checkpoints and performs inference on CPU. ImageNet normalization is applied inside the Transformer wrapper. Training weights carry the upstream NVIDIA research and evaluation license retained in docs/third_party/SEGFORMER_LICENSE.txt.

Training and reproducibility

All reported local models used three epochs, input width 160 and height 200, batch size 8 and seed 42. The CNN and frozen Transformer used learning rate 0.001; the fully fine tuned Transformer used 0.00006. Training ran on CPU using PyTorch 2.14.1 and Transformers 5.20.2.

The loss combines binary cross entropy with logits and soft Dice on positive documents. Authentic images contribute binary cross entropy without positive Dice. Positive pixel weighting is computed from the training masks and capped at 30 to limit unstable gradients.

The best checkpoint was selected using validation F1 at threshold 0.5. A validation only threshold search from 0.1 to 0.9 selected 0.9 for CNN and 0.8 for both Transformer variants. The test set was not used to select thresholds. All selected checkpoints are epoch 3.

Colab notebooks describe a planned 20 epoch, 320 by 400 experiment. That run has not been executed, and no result from it is claimed. Checkpoint configuration, history, validation thresholds and test predictions are supplied for independent inspection.

Measured results and interpretation

Global pixel metrics are calculated by aggregating true positives, false positives and false negatives over all 180 test images. Precision equals TP divided by TP plus FP; recall equals TP divided by TP plus FN; F1 equals 2TP divided by 2TP plus FP plus FN; IoU equals TP divided by TP plus FP plus FN.

The CNN had TP 4,193, FP 18,135 and FN 7,567. The fully fine tuned Transformer had TP 4,161, FP 20,449 and FN 7,599. The Transformer did not outperform the CNN on global pixel F1. A pretrained architecture alone does not guarantee improvement under limited training and low input resolution.

A separate document flag rule marks a page when predicted foreground occupies at least 0.1 percent of the model grid. It falsely flagged 90 of 90 authentic test pages for CNN and 77 of 90 for the fully fine tuned Transformer. The displayed region fraction is not a fraud probability.

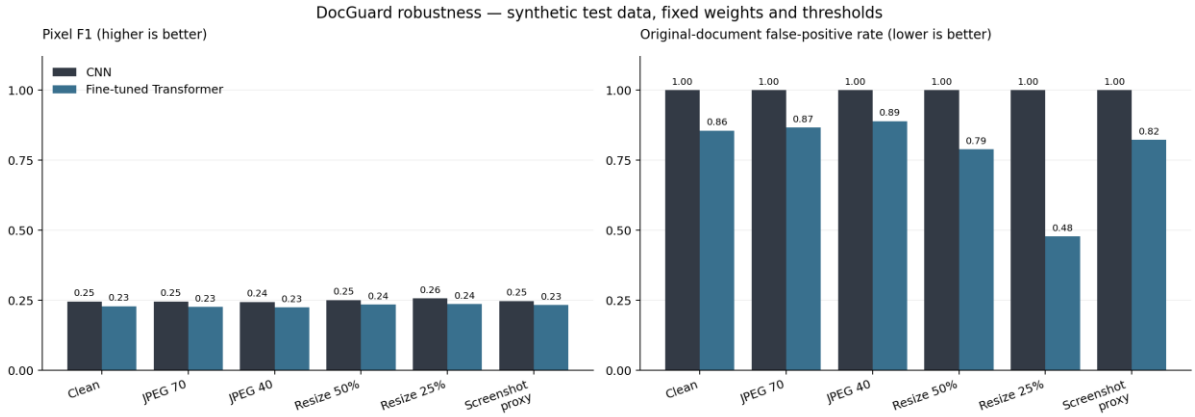
Model	Precision	Recall	F1	IoU
CNN	0.1878	0.3565	0.2460	0.1403
Frozen Transformer	0.1330	0.6283	0.2195	0.1233
Fine tuned Transformer	0.1691	0.3538	0.2288	0.1292

Robustness and CPU performance

Both application models were evaluated on all 180 test images under six fixed conditions: clean, JPEG quality 70, JPEG quality 40, 50 percent down and up resize, 25 percent down and up resize, and a screenshot proxy. Weights and thresholds were kept fixed; clean evaluation reproduced previous confusion counts.

CNN F1 across the six conditions was 0.2460, 0.2461, 0.2444, 0.2513, 0.2569 and 0.2472. Fully fine tuned Transformer F1 was 0.2288, 0.2269, 0.2252, 0.2352, 0.2364 and 0.2333. Small numerical gains under resizing do not establish reliability because false positive rates remain high.

The screenshot proxy applies 0.8 down and up resizing, blur 0.4 and JPEG quality 75. It is a simulation, not a real screenshot study. Median model forward time was 19.70 ms for CNN and 34.65 ms for Transformer using CPU, four threads, batch one, three warmups and twenty repetitions. These are not end to end application latencies.

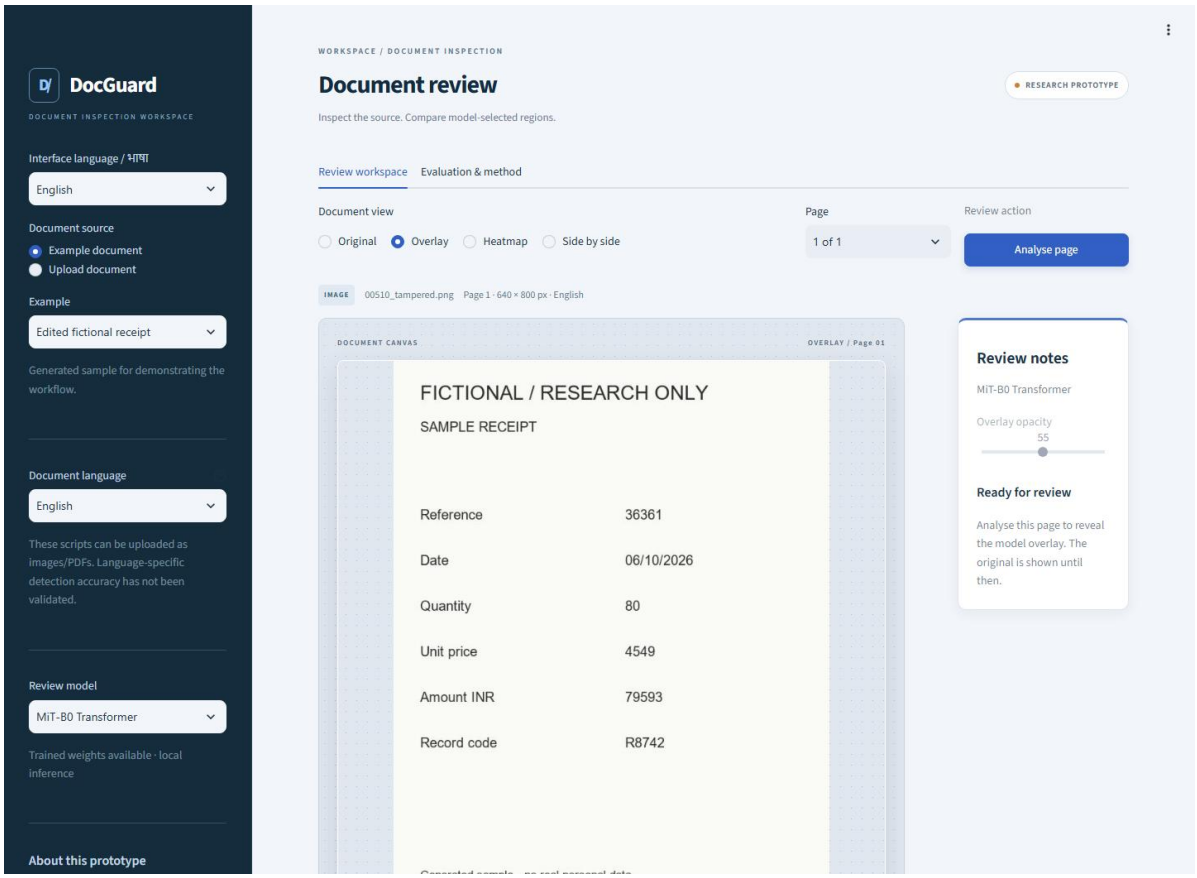


Application and regional languages

The Streamlit application supports image and PDF upload, page selection, model selection, heatmaps, overlays, side by side review and opacity adjustment. Review exports include PNG, an English PDF report and a ZIP with machine readable metadata.

The interface switches among English, Hindi, Marathi, Tamil and Bengali while preserving the current review. Document language metadata additionally includes Telugu, Urdu and Other. Hindi PDF and regional language receipt examples are included for demonstrating the workflow.

Language selection is not OCR, translation or validation of language specific model accuracy. Uploads are limited to 20 MB, images to 20 megapixels and PDFs to twenty pages. Selected PDF pages are rendered with a maximum side of 1,600 pixels. Uploaded documents remain in the local application process.



Verification and demonstration

Fourteen unit checks passed, including metric calculations, model restore, image and PDF constraints, export behavior and interface language state. Browser verification exercised fourteen application flows, and seven additional language checks covered retained selections and mobile overflow.

For a demonstration, launch the app, select an edited English example, choose CNN, analyse the page, inspect the overlay and export a report. Repeat with the Transformer to explain differences. Upload the supplied Hindi PDF, switch the interface language and demonstrate preserved review state.

The demonstration should end with an authentic sample and the measured false positive limitation. A highlighted patch is a model suggestion requiring review. The app must not be presented as a validated fraud certification system.

Limitations and future work

The current experiment has synthetic domain bias, small model input resolution, three training epochs and very high false positive rates. Pixel labels are rectangular, and visual evidence may be absent for cleanly replaced fields. No real document, OCR, forgery type or language specific benchmark is available.

Next steps are a licensed real document benchmark, realistic mask annotations, longer controlled training, higher resolution, explicit calibration and independent document level validation. These require new measurements; improvements are proposals rather than achieved results.

DocGuard demonstrates the complete engineering workflow from synthetic data to CNN and Transformer comparison, robustness experiments and a multilingual local app. Its strongest deliverable is reproducible experimentation with an honest account of weak detection performance.

References and project evidence

Ronneberger, Fischer and Brox. U Net Convolutional Networks for Biomedical Image Segmentation. 2015. https://arxiv.org/abs/1505.04597

Xie and colleagues. SegFormer Simple and Efficient Design for Semantic Segmentation with Transformers. 2021. https://arxiv.org/abs/2105.15203

NVIDIA MiT B0 pretrained encoder model card. https://huggingface.co/nvidia/mit-b0

DocTamper official repository, considered but not used. https://github.com/qcf-568/DocTamper

Local primary evidence: runs/*/config.json, history.json, validation.json and test_metrics.json; deliverables/model_comparison.json; deliverables/robustness/results.json; deliverables/app_preview/browser_checks.json and language_checks.json. Repository: https://github.com/Shrashti1234/Docguard